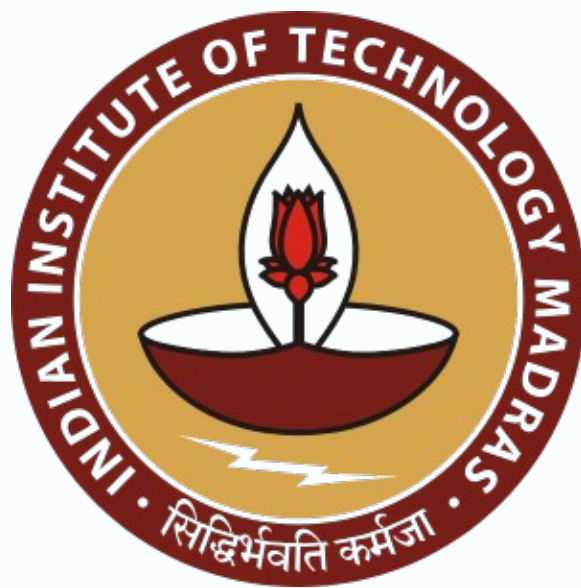


CS3205: INTRODUCTION TO COMPUTER NETWORKS

ASSIGNMENT 2



KARTHIK KASHYAP K

EE23B030

CS3205: Introduction to Computer Networks

ASSIGNMENT 2: PERFORMANCE REPORT

KARTHIK KASHYAP K

Contents

0	Introduction	3
1	Methodology	3
1.1	Workloads Used	3
1.2	Metrics Collected	3
1.3	Execution Commands	4
1.4	Run Set Used for This Report	4
2	Test Results with Graphs	4
2.1	Generated Artifacts	4
2.2	Load Test Summary (10 clients, 20 messages/client)	5
2.3	Stress Test Highlights (5 to 60 clients)	5
3	Analysis and Observations	5
4	Bottlenecks Identified	6
5	Potential Optimizations	6
6	Self Improvements	6

§0 Introduction

Since the codes and images are best viewed externally than in a pdf file, I have also attached the respective codes and images used in this assignment in this external site: [Assignment 2 Files](#). This also contains a small amount of info that wouldn't fit into this assignment file, but need mentioning (but can otherwise be ignored).

Also, a good amount of LLMs (ChatGPT) were used in this assignment since I am not very versed with c language, and this assignment was lengthy and I couldn't rely on learning the core of the language given the amount of time it would take.

§1 Methodology

The benchmarking methodology compares three server implementations:

1. `chat_server_thread` (thread-per-client)
2. `chat_server_fork` (process-per-client)
3. `chat_server_select` (single-process select-based)

§1.1 Workloads Used

1. Load Test (`tests/load_test.sh`)

- Fixed concurrent clients
- Fixed messages/client

2. Stress Test (`tests/stress_test.sh`)

- Gradually increasing client count
- Stop condition based on failure ratio

§1.2 Metrics Collected

1. Server Metrics (in-server monitor module)

- CPU usage (%)
- VmRSS (KB)
- PSS (KB)
- Logged to `chat_monitor.csv`

2. Client Metrics

- Message delivery latency in microseconds (`LATENCY_US`)
- Derived summary: sample count, avg, p50/p95, min, max

§1.3 Execution Commands

Load tests:

```
./tests/load_test.sh thread 10 20 1000
./tests/load_test.sh fork 10 20 1000
./tests/load_test.sh select 10 20 1000
```

Stress tests:

```
./tests/stress_test.sh thread 5 5 60 10 500 0.10
./tests/stress_test.sh fork 5 5 60 10 500 0.10
./tests/stress_test.sh select 5 5 60 10 500 0.10
```

Graph generation:

```
./scripts/plot_latest.sh
```

§1.4 Run Set Used for This Report

Load Runs

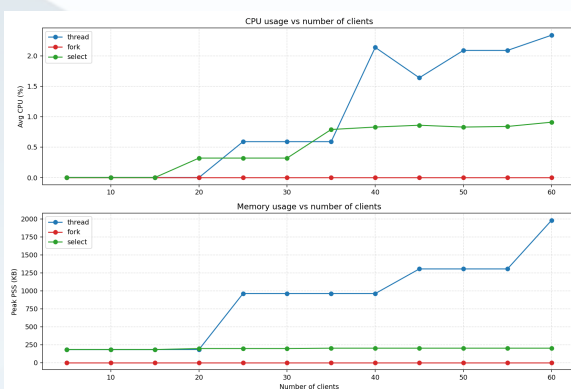
- results/load_thread_20260303_221547
- results/load_fork_20260303_221556
- results/load_select_20260303_221605

Stress Runs

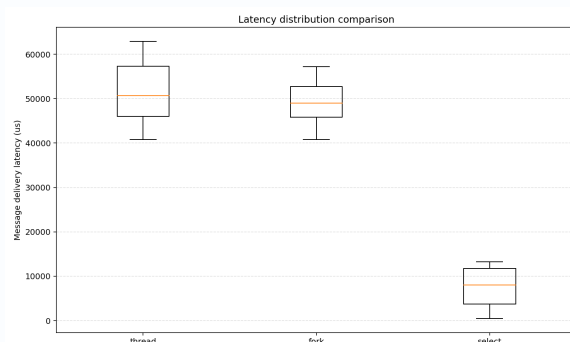
- results/stress_thread_20260303_221639
- results/stress_fork_20260303_221654
- results/stress_select_20260303_221703

§2 Test Results with Graphs

§2.1 Generated Artifacts



CPU Memory vs Clients



Latency Distribution Comparison

§2.2 Load Test Summary (10 clients, 20 messages/client)

Mode	Success/Fail	Duration (ms)	Throughput (msg/s)	Avg Lat (us)	P95 Lat (us)	VmRSS (KB)	PSS (KB)
Thread	10 / 0	229	873.36	53295.98	62179	1852	187
Fork	10 / 0	229	873.36	50856.28	56481	1892	136
Select	10 / 0	271	738.01	13499.48	41726	1848	183

§2.3 Stress Test Highlights (5 to 60 clients)

Thread

- 60 clients: throughput 463.32 msg/s
- Avg latency 84701.91 us, p95 185434 us
- Peak memory: VmRSS 3896 KB, PSS 1981 KB

Fork

- 60 clients: throughput 2419.35 msg/s
- Avg latency 126945.97 us, p95 204934 us

Select

- 60 clients: throughput 448.10 msg/s
- Avg latency 29692.20 us, p95 62179 us
- Peak memory: VmRSS 1932 KB, PSS 202 KB

§3 Analysis and Observations

1. Thread Model

- Good at low-to-mid load, but latency tails become large at higher client counts.
- At 60 clients, latency increases substantially (avg 84.7ms, p95 185ms).
- Degradation starts around 25–30 clients (avg latency crosses 100 ms), with unstable runtime spikes at 40+ clients (duration jumps to 1260 ms at 40 and 50 clients).

2. Fork Model

- Highest throughput in this test set during stress (up to 2419 msg/s at 60 clients).
- Also shows highest latency at scale (avg 127ms, p95 205ms), indicating throughput-latency tradeoff.
- Throughput scales well up to 60 clients, but latency degradation appears from around 25 clients onward (avg latency 100+ ms, p95 near/above 200 ms at high load).

3. Select Model

- Best latency behavior in load run (avg 13.5ms, p95 41.7ms).
- Throughput scales more slowly than fork under high concurrency in this implementation.
- Memory footprint remains comparatively low and stable.
- Major degradation begins around 35 clients (duration jumps from 345 ms at 30 clients to 1284 ms at 35 clients), while latency remains lower than thread/fork at high load.

§4 Bottlenecks Identified

1. Broadcast fan-out is linear in number of clients.
2. String parsing and formatting overhead increases CPU usage.
3. Fork mode incurs process creation and management overhead.
4. Select mode centralizes work in a single event loop.
5. Monitoring visibility for fork stress runs requires improvement.

§5 Potential Optimizations

1. Replace linear scans with indexed structures (hash-based lookup).
2. Move from `select` to `poll`/`epoll`.
3. Reduce text parsing overhead using compact structured formats.
4. Introduce buffered/asynchronous send queues.
5. Improve synchronization granularity in thread mode.

§6 Self Improvements

A terminal-based chat UI was implemented for interactive usage and demonstration. It supports:

- conversation sidebar (broadcast + private contacts)
- channel switching
- status and history commands

Detailed usage and commands are documented in `README.md`.

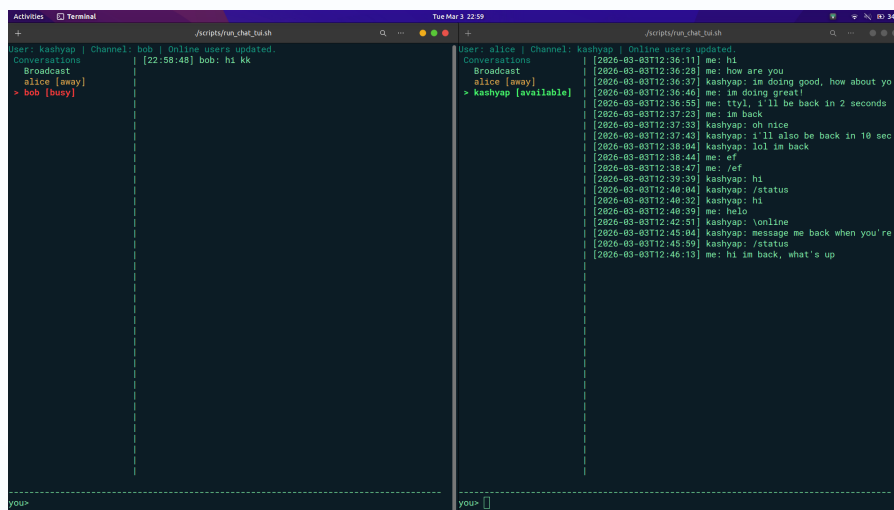


Figure 2: Chat Terminal UI